

PRUEBA PERIFERIA

Documentación de Instalación y Arquitectura

Backend Spring Boot · Frontend Angular 17 · PostgreSQL · Docker

1. Descripción general del proyecto

Prueba Periferia es un sistema de gestión de publicaciones desarrollado para Periferia IT Group. Permite a usuarios autenticados crear, editar, listar y eliminar publicaciones organizadas por categorías, con control de acceso basado en JSON Web Tokens (JWT).

La solución está compuesta por tres partes que se integran y despliegan de forma conjunta:

- Backend: API REST en Java / Spring Boot, responsable de la lógica de negocio, autenticación y persistencia de datos.
- Frontend: aplicación de una sola página (SPA) en Angular 17, que consume la API REST y ofrece la interfaz de usuario.
- Base de datos: PostgreSQL, donde se almacenan usuarios, categorías, publicaciones y su información de auditoría.

Toda la solución se empaqueta y despliega mediante Docker y Docker Compose, de modo que un mismo comando levanta los tres servicios integrados, sin necesidad de instalar Java, Node.js o PostgreSQL en la máquina donde se ejecuta.

Arquitectura de despliegue

Frontend Angular 17 → Nginx → Backend API (Spring Boot) → PostgreSQL

El contenedor del frontend sirve los archivos estáticos del build de Angular a través de Nginx, y ese mismo Nginx actúa como proxy inverso: reenvía internamente las peticiones a la API (/api/v1/) hacia el contenedor del backend. Así, el navegador solo se comunica con un único origen, lo que evita tener que configurar CORS para el flujo normal de la aplicación.

2. Funcionalidades detalladas

2.1 Autenticación y sesión

- Login: POST /api/v1/api/auth/login recibe username y password, valida contra AuthenticationManager (BCrypt) y devuelve accessToken + refreshToken (JWT).

- Validación de sesión: GET /api/v1/api/auth/me devuelve los datos del usuario dueño del token actual (id, username, activo, tipoDocumento, numeroDocumento); sin token válido responde 401.
- Renovación: POST /api/v1/api/auth/refresh rota accessToken y refreshToken a partir de un refreshToken vigente, sin pedir credenciales de nuevo. Existe una variante legacy (/auth/refresh2) mantenida por compatibilidad; se recomienda usar solo /refresh.
- Las contraseñas nunca se devuelven en las respuestas ni se almacenan en texto plano (BCrypt).
- En el frontend, AuthApiService persiste los tokens en localStorage, authInterceptor adjunta automáticamente el header Authorization a cada petición saliente, y authGuard protege las rutas privadas (redirige a login si no hay sesión, y evita reingresar a login si ya la hay).

2.2 Gestión de usuarios

- CRUD completo bajo /api/usuarios (requiere JWT): listar, obtener por id, crear, editar y eliminar.
- La creación hashlea la contraseña con BCrypt; activo y eliminado toman valores por defecto si no se envían.
- La edición es parcial: solo los campos no nulos del body se aplican, y la contraseña solo se re-hashlea si se envía un valor nuevo.
- El borrado es lógico: un trigger de base de datos intercepta el DELETE físico y marca la fila como eliminada en lugar de borrarla, preservando la integridad referencial con las publicaciones existentes del usuario.

2.3 Perfiles (roles)

- Catálogo de roles bajo /api/perfiles, con relación N:M hacia usuario a través de la tabla usuario_perfil.
- CRUD estándar; a diferencia de usuario y publicacion, el borrado de un perfil es físico, y la tabla usuario_perfil tiene ON DELETE CASCADE hacia perfil.

2.4 Categorías

- Catálogo bajo /api/categorias, con nombre y slug únicos.
- Borrar una categoría no elimina las publicaciones asociadas: publicacion.categoria_id usa ON DELETE SET NULL, solo se desvinculan.
- El frontend consume este listado para poblar el selector de categoría en el formulario de publicaciones.

2.5 Gestión de publicaciones (núcleo funcional)

- CRUD bajo /api/publicaciones, con filtro por tipo vía query param: 0 = todas (por defecto), 1 = solo las del usuario autenticado, 2 = las de otros usuarios.
- Autoría automática: si el body no incluye el usuario, se asigna automáticamente el dueño del token JWT que hace la petición.
- El slug se autogenera a partir del título si no se envía explícitamente.
- Estado de la publicación: 0 Borrador, 1 Publicado, 2 Archivado.
- El borrado es lógico, con el mismo mecanismo de trigger que usuario.
- En el frontend, el listado se muestra en una tabla ag-Grid con paginación, orden y filtro por tipo; alta y edición se hacen mediante un diálogo modal con formulario reactivo tipado (título, resumen, contenido, estado, categoría, fecha de publicación); el borrado pide confirmación en un diálogo aparte.
- El estado de la lista (ítems, carga, error, filtro activo) se centraliza en un store reactivo (signalStore) que recarga automáticamente tras cada alta, edición o borrado.

2.6 Interfaz y experiencia de usuario

- Shell de la aplicación con barra superior (menú, selector de idioma, alternador de tema claro/oscuro persistido en localStorage, cierre de sesión), panel lateral y pie de página.
- Internacionalización parcial vía ngx-translate (español/inglés en algunas secciones; el resto de las pantallas tiene los textos embebidos directamente en español).
- Manejo de errores HTTP centralizado: cada llamada a la API tipa el error como `HttpErrorResponse` y los componentes muestran al usuario un mensaje legible en vez del error crudo.

2.7 Manejo de errores del backend

Todas las excepciones de dominio se centralizan en un manejador global y se traducen a respuestas RFC 7807 (ProblemDetail) con el código HTTP correcto:

Situación	HTTP
Usuario, categoría o publicación inexistente	404 Not Found
Token o refresh token inválido, expirado, o usuario inactivo	401 Unauthorized
Datos de entrada faltantes o mal formados	400 Bad Request
Username o slug duplicado / violación de restricción de datos	409 Conflict
Falla de validación sobre un campo del formulario/DTO	400 Bad Request (detalle por campo)
Login fallido o petición sin autenticar	401 Unauthorized
Sin permisos para la operación	403 Forbidden
Cualquier otro error no controlado	500 Internal Server Error (detalle solo en el log del servidor)

3. Tecnologías utilizadas

3.1 Backend

Componente	Tecnología
Lenguaje	Java 17
Framework	Spring Boot 3.4.2 (spring-boot-starter-parent)
Capa web	Spring Web (MVC)
Seguridad	Spring Security (stateless) + JWT (io.jsonwebtoken / jjwt 0.11.5)
Persistencia	Spring Data JPA + Hibernate
Base de datos	PostgreSQL (runtime) · H2 en memoria (tests de contexto)
Documentación de API	springdoc-openapi (Swagger UI)
Exportación de archivos	JSch (com.github.mwiede:jsch) vía SSH
Utilidades	Lombok, Gson
Cobertura de tests	JaCoCo

Tests de integración	Testcontainers (PostgreSQL real, requiere Docker)
Empaquetado	Maven (wrapper mvnw incluido)

Autenticación: el backend expone POST /api/v1/api/auth/login (usuario y contraseña) y devuelve un accessToken y un refreshToken. Las rutas protegidas exigen el header Authorization: Bearer {accessToken}. El refreshToken permite renovar la sesión mediante POST /api/v1/api/auth/refresh sin pedir credenciales de nuevo. Las contraseñas se almacenan cifradas con BCrypt.

3.2 Frontend

Componente	Tecnología
Framework	Angular 17 (standalone components)
Interfaz de usuario	Angular Material 17, Angular CDK, Bootstrap 5, Bootstrap Icons, Font Awesome
Gestión de estado	@ngrx/signals (signalStore)
Datos tabulares	ag-grid-angular / ag-grid-community
Comunicación HTTP	HttpClient + interceptor funcional para adjuntar el JWT
Internacionalización	@ngx-translate/core + @ngx-translate/http-loader
Notificaciones	ng-angular-popup + MatSnackBar
Testing	Karma + Jasmine
Lint / formato	ESLint (@angular-eslint) + Prettier

El frontend nunca hardcodea la URL del backend: cada entorno (local, develop, qa, production, docker) tiene su propio archivo environment.*.ts con la configuración de la API, seleccionado automáticamente según la configuración de build de Angular (fileReplacements en angular.json).

Patrones aplicados: componentes standalone (sin NgModules de feature), formularios reactivos tipados (NonNullableFormBuilder, evita el error clásico de T | null en cada control), interceptor HTTP funcional (en vez de la clase clásica HttpInterceptor) y guards funcionales para proteger rutas.

3.3 Base de datos

Componente	Detalle
Motor	PostgreSQL 16 (imagen oficial postgres:16-alpine)
Inicialización	Script SQL (db/init/01-schema.sql), ejecutado solo en el primer arranque del volumen
Persistencia	Volumen Docker nombrado (periferia_pgdata), sobrevive a reinicios de los contenedores
Entidades principales	usuario, perfil, usuario_perfil, categoria, publicacion, publicacion_adjunto

Borrado	Lógico (columna eliminado) para usuario y publicacion, reforzado con CHECK constraints
---------	--

3.4 Docker e infraestructura

Componente	Detalle
Orquestación	Docker Compose (3 servicios: db, backend, frontend)
Imagen base backend (build)	maven:3.9.9-eclipse-temurin-17
Imagen base backend (runtime)	eclipse-temurin:17-jre-jammy (usuario no root)
Imagen base frontend (build)	node:20-alpine
Imagen base frontend (runtime)	nginx:1.27-alpine
Imagen base de datos	postgres:16-alpine
Red interna	periferia-net (bridge), comunicación entre contenedores por nombre de servicio
Healthchecks	db (pg_isready), backend (TCP :8080) — frontend depende de que backend esté "healthy"
Persistencia	Volumen nombrado periferia_pgdata para los datos de PostgreSQL

3.5 Calidad y pruebas

- Backend: tests unitarios/de contexto con H2 en memoria (no requieren nada externo); tests de integración con Testcontainers levantando un PostgreSQL real, para validar comportamiento específico de triggers y funciones de base de datos (requieren Docker en la máquina donde se ejecutan).
- Cobertura del backend generada con JaCoCo en cada ejecución de mvn test.
- Frontend: linting con ESLint (incluye reglas de accesibilidad @angular-eslint/template) y formateo con Prettier; base de tests unitarios con Karma + Jasmine.
- Documentación de API autogenerada con springdoc-openapi (Swagger UI), navegable en caliente contra el backend en ejecución.

4. Instalación y puesta en marcha

El despliegue completo de la solución se realiza con Docker y Docker Compose, sin necesidad de instalar Java, Node.js, Maven ni PostgreSQL en la máquina anfitriona.

4.1 Requisitos previos

- Docker Desktop instalado y en ejecución (imprescindible: incluye el motor de Docker y Docker Compose v2). Descarga: <https://www.docker.com/products/docker-desktop> — en Windows, con el backend WSL2 habilitado. Sin Docker Desktop abierto y corriendo, docker compose build / docker compose up fallan con un error de conexión al daemon de Docker.
- Puertos libres en el equipo: 5432 (PostgreSQL), 8080 (backend) y 8090 (frontend). Son configurables si están ocupados.

- Repositorio del proyecto clonado o copiado localmente, con las carpetas pruebaPeriferiaBack y pruebaPeriferiaFront dentro de la raíz del proyecto.

4.2 Configuración de variables de entorno

En la raíz del proyecto existe un archivo `.env.example` con la plantilla de variables necesarias. Se debe copiar a `.env` y ajustar los valores según el entorno:

```
cp .env.example .env
```

Variable	Uso	Valor por defecto
DB_USER	Usuario de PostgreSQL	postgres
DB_PASSWORD	Contraseña de PostgreSQL	root
DB_EXPOSED_PORT	Puerto de PostgreSQL publicado en el host	5432
BACKEND_EXPOSED_PORT	Puerto del backend publicado en el host	8080
JWT_SECRET	Clave secreta para firmar los JWT	valor de ejemplo — cambiar en un entorno real
JWT_EXPIRATION	Expiración del access token en milisegundos	3600000 (1 hora)
FRONTEND_EXPOSED_PORT	Puerto del frontend (Nginx) publicado en el host	8090

El archivo `.env` contiene valores sensibles y está excluido del repositorio mediante `.gitignore`; nunca debe subirse a control de versiones.

4.3 Construcción y arranque de los contenedores

Desde la raíz del proyecto (donde está `docker-compose.yml`):

```
# Construir las imágenes de backend y frontend
docker compose build

# Levantar los 3 servicios en segundo plano
docker compose up -d

# Verificar el estado de los contenedores
docker compose ps
```

4.4 Acceso a la aplicación

Aplicación (frontend): <http://localhost:8090>

Documentación interactiva de la API (Swagger UI): <http://localhost:8080/api/v1/swagger-ui.html>

El flujo normal de uso es siempre a través del puerto del frontend (8090): el navegador no necesita llamar directamente al backend, ya que Nginx hace de intermediario para las peticiones a la API.

4.5 Logs, reconstrucción y detención

```
# Ver logs de todos los servicios
docker compose logs -f

# Ver logs de un servicio puntual
docker compose logs -f frontend
docker compose logs -f backend
docker compose logs -f db

# Reconstruir un servicio tras cambios en su código
docker compose up -d --build frontend
docker compose up -d --build backend

# Detener los contenedores (conserva los datos de la base de datos)
docker compose down

# Detener y borrar también el volumen de datos de PostgreSQL
docker compose down -v
```

4.6 Desarrollo local sin Docker (opcional)

Para trabajar en el día a día sin reconstruir imágenes en cada cambio, cada parte puede levantarse por separado:

Backend: `./mvnw spring-boot:run` (requiere PostgreSQL accesible en `localhost:5432`, o el contenedor `db` levantado con `docker compose up -d db`).

Frontend: `npm install` seguido de `ng serve` (por defecto apunta a `http://localhost:8080` según `environment.ts`).

5. Colección Postman

El repositorio incluye una colección de Postman lista para consumir todos los endpoints del backend, generada directamente a partir de los controllers, la configuración de seguridad y las entidades JPA — no es una lista inventada, refleja el contrato real de la API.

Ubicación en el repositorio: `pruebaPeriferiaBack/Claude`
`outputs/pruebaPeriferia.postman_collection.json`

5.1 Estructura de la colección

- Auth: Login, Me (validar token), Refresh y Refresh2 (variante legacy).
- Test: Status — healthcheck público, sin autenticación.
- Usuarios: listar, obtener por id, crear, editar, eliminar.
- Perfiles: listar, obtener por id, crear, editar, eliminar (catálogo de roles).
- Categorías: listar, obtener por id, crear, editar, eliminar.
- Publicaciones: listar (con filtro por tipo), obtener por id, crear, editar, eliminar.

5.2 Variables de colección

Variable	Uso
baseUrl	URL base de la API. Por defecto <code>http://localhost:8080/api/v1</code> (backend directo).
baseUrlDocker	Alternativa para probar a través del proxy de Nginx del frontend dockerizado: <code>http://localhost:8090/api/v1</code> . Copiar su valor a <code>baseUrl</code> para usarla.
username / password	Credenciales para la petición de Login.
accessToken / refreshToken	Se completan automáticamente al ejecutar Login o Refresh — no requieren edición manual.
userId / perfilId / categoryId / publicacionId	Identificadores usados en las rutas con <code>{id}</code> de cada módulo.

5.3 Flujo de uso recomendado

- Completar `username` y `password` en las variables de la colección (o en el Environment de Postman).
- Ejecutar Auth → Login: la colección trae un script de test que captura `accessToken` y `refreshToken` de la respuesta y los guarda como variables de colección automáticamente.
- A partir de ahí, cualquier request de Usuarios, Perfiles, Categorías o Publicaciones ya envía el header `Authorization: Bearer {{accessToken}}` sin configuración adicional, gracias a la autenticación heredada a nivel de colección.
- Si el `accessToken` expira, ejecutar Auth → Refresh (usa el `refreshToken` guardado) para renovar ambos tokens sin volver a iniciar sesión.
- Para probar contra la solución dockerizada completa en vez del backend suelto, cambiar el valor de `baseUrl` por el de `baseUrlDocker`.

6. Solución de problemas comunes

Errores de CORS al iniciar sesión: no deberían aparecer accediendo por `http://localhost:8090`, ya que todo el tráfico es same-origin gracias al proxy de Nginx. Si aparecen, verificar que se está entrando por el puerto del frontend y no llamando directamente a `localhost:8080` desde otro origen.

Errores 401 / 403: revisar que el `accessToken` se esté enviando en el header `Authorization` y que no haya expirado (`JWT_EXPIRATION`). Si expiró, debe usarse el endpoint de refresh; si el `refreshToken` también expiró, es necesario iniciar sesión de nuevo.

Puerto ya en uso: cambiar `DB_EXPOSED_PORT`, `BACKEND_EXPOSED_PORT` o `FRONTEND_EXPOSED_PORT` en `.env` por un puerto libre y volver a ejecutar `docker compose up -d`.

Error 404 en rutas de Angular al refrescar la página: dentro de Docker ya está resuelto mediante la configuración `try_files` de Nginx, que redirige cualquier ruta no encontrada al `index.html` de la aplicación Angular.

Fallos de npm ci al construir la imagen del frontend: ocurre cuando `package-lock.json` no está sincronizado con `package.json`. Se soluciona regenerando el lock file (`rm -rf node_modules package-lock.json && npm install`) antes de reconstruir la imagen.

La base de datos no tiene las tablas esperadas: el script de inicialización solo se ejecuta la primera vez que el volumen de datos está vacío. Usar `docker compose down -v` para forzar un volumen limpio y que el script se ejecute de nuevo.